VECTOR

# MICROSAR CAN transceiver driver

## Technical Reference

TJA1040

Version 4.2.2


| | |
|---|---|
| Authors | Matthias Fleischmann, Senol Cendere, Mihai Olariu, Timo Vanoni, Eugen Stripling, Andreas Weinrauch, Jan Hammer |
| Status | Released |

# Document Information

## History

| Author | Date | Version | Remarks |
|---|---|---|---|
| Matthias Fleischmann | 2008-04-21 | 1.00 | Creation |
| Senol Cendere | 2008-06-10 | 1.01 | Rework file names<br>Rework tool description |
| Mihai Olariu | 2008-12-15 | 1.02 | Add support for the platforms which cannot wake-up by CAN bus activity |
| Timo Vanoni | 2009-02-10 | 1.03 | Added Tja1040 specific information |
| Timo Vanoni | 2009-09-11 | 1.04 | Added information about AMIS43665.<br>Updated description for ICU notification function and GENy configuration. |
| Timo Vanoni | 2010-04-21 | 1.05 | Added API CanTrcv_30_<Your_Trcv>_Wait()<br>Change of initialization flow. |
| Timo Vanoni | 2011-02-22 | 1.06 | Rename files, add support for Tja1042 |
| Timo Vanoni | 2011-04-28 | 1.07 | Add support for Identity Manager Configurations |
| Eugen Stripling | 2011-09-05 | 2.00.00 | Implementation according to ASR3.2.1 |
| Eugen Stripling | 2012-01-18 | 2.01.00 | Adapted according to fix of ESCAN00055237 |
| Timo Vanoni | 2012-08-15 | 3.00.00 | Add support for ASR4.0.3 |
| Eugen Stripling | 2013-05-21 | 3.00.01 | Serial release for ASR4.0.3 |
| Timo Vanoni | 2014-02-07 | 3.00.02 | Add support for Tja1049 |
| Eugen Stripling | 2014-10-22 | 4.00.00 | AUTOSAR3 removed, Post-build selectable |
| Eugen Stripling | 2015-02-10 | 4.00.01 | Add support for TI HVDA553-Q1 |
| Eugen Stripling | 2015-04-09 | 4.00.02 | Add support for Tja1044 |
| Eugen Stripling | 2015-07-29 | 4.00.03 | Add support for: R2A25416SP, NCV7342 |
| Eugen Stripling | 2015-12-11 | 4.01.00 | Chapter 3.1.1.3 added, chapter 3.2 adapted, Support for Microchip MCP2561/2FD added |
| Eugen Stripling | 2016-01-27 | 4.01.01 | Add support for MC33901WEF, NCV7349, TLE7251 |
| Andreas Weinrauch | 2016-08-22 | 4.01.02 | Add support for TI HVDA551-Q1 |
| Andreas Weinrauch | 2017-01-11 | 4.01.03 | Rework because of review findings |
| Jan Hammer | 2017-02-21 | 4.01.04 | Add support for TJA1046, TLE6251D |
| Jan Hammer | 2017-03-08 | 4.01.05 | Add support for TJA1059 |
| Jan Hammer | 2017-03-20 | 4.01.06 | Add support for TI TCAN1042v-Q1 |
| Jan Hammer | 2017-04-18 | 4.01.07 | Add support for TI SN65HVD1040-Q1 |

| | | | |
|---|---|---|---|
| Eugen Stripling | 2017-08-23 | 4.02.00 | Update to newest template, SafeBSW support added, all supported and known derivatives added<br>Add support for TLE9251VLE, TLE9251VSJ |
| Jan Hammer | 2017-10-25 | 4.02.01 | Add support for TI SN65HVD251 |
| Jan Hammer | 2018-05-03 | 4.02.02 | Add support for TLE8251VSJ, NCV7344 |

## Reference Documents

| No. | Source | Title | Version |
|---|---|---|---|
| [1] | AUTOSAR | AUTOSAR_SWS_CAN_TransceiverDriver.pdf | 3.0.0 |
| [2] | AUTOSAR | AUTOSAR_SWS_DET.pdf | 2.2.1 |
| [3] | AUTOSAR | AUTOSAR_BasicSoftwareModules.pdf | V1.0.0 |
| [4] | NXP | TJA1040.pdf | 2003 Feb 19 |
| [5] | ON Semiconductor | AMIS-42665-D.pdf | January, 2013 − Rev. 10 |
| [6] | NXP | TJA1042.pdf | Rev. 9 — 23 May 2016 |
| [7] | TI | sn65hvda541-q1.pdf | SLLS804B – MARCH 2009– REVISED SEPTEMBER 2010 |
| [8] | NXP | TJA1049_Rev5.pdf | Rev. 5 — 23 May 2016 |
| [9] | TI | hvda553-q1.pdf | SLLSEC4 –JUNE 2013 |
| [10] | NXP | TJA1044_Rev5.pdf | Rev. 5 — 23 May 2016 |
| [11] | Renesas | rej03f0289_0101_R2A25416SP.PDF | Rev.1.01, Feb.04.2010 |
| [12] | ON Semiconductor | NCV7342-D.pdf | June, 2016 − Rev. 4 |
| [13] | Microchip | MCP2561_2_FD.pdf | DS20005284A |
| [14] | NXP | MC33901.pdf | Rev. 3.0, 6/2015 |
| [15] | ON Semiconductor | NCV7349-D.PDF | December, 2014 − Rev. 1 |
| [16] | Infineon | Infineon-TLE7251V-DS-v01_00-EN.pdf | Rev. 1.0, 2015-09-10 |
| [17] | TI | hvda551-q1.pdf | SLLSEC4 –JUNE 2013 |
| [18] | NXP | TJA1046_2.pdf | Rev. 2 — 23 May 2016 |

| | | | |
|---|---|---|---|
| [19] | Infineon | Infineon-TLE6251D-DS-v01_10-EN.pdf | Rev. 1.1 2016-06-06 |
| [20] | NXP | TJA1059.pdf | Rev. 3 — 23 May 2016 |
| [21] | TI | tcan1042v-q1.pdf | SLLSES9A – FEBRUARY 2016– REVISED MAY 2016 |
| [22] | TI | sn65hvd1040-q1_revE_2016_09.pdf | SLLS753E – FEBRUARY 2007– REVISED SEPTEMBER 2016 |
| [23] | NXP | TJA1902A_Datasheet.pdf | Rev. 1.00 — 2 September 2016 |
| [24] | Infineon | TLE9251V_TDS_04_Infineon.pdf | Rev. 0.4, 2016-01-26 |
| [25] | TI | SN65HVD251.pdf | SLLS545B– NOVEMBER 2002– REVISED SEPTEMBER 2003 |
| [26] | Infineon | Infineon-TLE8251VSJ-DS-v01_00-EN.pdf | Rev. 1.0, 2016-07-15 |
| [27] | ON Semiconductor | NCV7344-D.PDF | February, 2017 − Rev. 1 |

Scope of the Document

This technical reference describes the general use of the CAN transceiver driver basis software.



**Caution**
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

## Contents

**1 Component History ..... 8**

**2 Introduction ..... 9**
2.1 Architecture Overview ..... 10
2.2 Supported Hardware ..... 11

**3 Functional Description ..... 13**
3.1 Features ..... 13
3.1.1 Deviations ..... 13
3.1.1.1 Notification functions ..... 13
3.1.1.2 Unused BSWMD parameters ..... 14
3.1.1.3 Initialization ..... 14
3.1.2 Additions / Extensions ..... 14
3.1.3 Limitations ..... 14
3.1.3.1 Support of SPI ..... 14
3.2 Initialization ..... 15
3.3 Operating modes ..... 15
3.3.1 Setting of operating mode ..... 17
3.3.2 Getting of operating mode ..... 17
3.4 Main Functions ..... 17
3.5 Get version info ..... 17
3.6 Wake-up by bus event detection ..... 18
3.6.1 Wake-up reason ..... 18
3.6.2 Wake-up mode ..... 18
3.7 Error Handling ..... 18
3.7.1 Development Error Reporting ..... 18

**4 Integration ..... 20**
4.1 Scope of Delivery ..... 20
4.1.1 Static Files ..... 20
4.1.2 Dynamic Files ..... 20
4.2 Critical Sections ..... 20
4.2.1 CANTRCV_EXCLUSIVE_AREA_0 ..... 20
4.3 Compiler Abstraction and Memory Mapping ..... 21

**5 Dependencies to other components ..... 22**
5.1 DIO driver ..... 22
5.2 ICU driver ..... 22
5.2.1 ICU configuration ..... 23

5.2.2 Implementation of the signal notification function ..... 24

**6 API Description ..... 25**
6.1 Services provided by CANTRCV ..... 25
6.1.1 CanTrcv_30_Tja1040_InitMemory ..... 25
6.1.2 CanTrcv_30_ Tja1040_Init ..... 26
6.1.3 CanTrcv_30_Tja1040_SetOpMode ..... 27
6.1.4 CanTrcv_30_ Tja1040_GetOpMode ..... 28
6.1.5 CanTrcv_30_ Tja1040_GetBusWuReason ..... 29
6.1.6 CanTrcv_30_ Tja1040_SetWakeupMode ..... 30
6.1.7 CanTrcv_30_ Tja1040_GetVersionInfo ..... 31
6.1.8 CanTrcv_30_Tja1040_CheckWakeup ..... 32
6.1.9 CanTrcv_30_Tja1040_MainFunction ..... 33
6.2 Services used by CANTRCV ..... 33

**7 Configuration ..... 34**
7.1 Configuration with DaVinci Configurator 5 ..... 34
7.2 Configuration Variants ..... 34
7.3 Configuration manually in Header Files ..... 34
7.3.1 CANTRCV_30_TJA1040_IMPLEMENT_APPL_CHECK_WA ..... 34

**8 Glossary and Abbreviations ..... 36**
8.1 Glossary ..... 36
8.2 Abbreviations ..... 36

**9 Contact ..... 37**

## Illustrations

Figure 2-1 AUTOSAR 4.2 Architecture Overview ..... 10
Figure 2-2 Interfaces to adjacent modules of the CANTRCV ..... 11
Figure 3-1 Operating mode state machine ..... 16
Figure 5-1 Add an ICU channel ..... 23
Figure 5-2 ICU channel configuration for wake-up via transceiver ..... 23
Figure 5-3 ICU wake-up capability ..... 24

## Tables

Table 1-1 Component history ..... 8
Table 3-1 Supported AUTOSAR standard conform features ..... 13
Table 3-2 Not supported AUTOSAR standard conform features due to hardware limitation ..... 13
Table 3 Deviation of APIs used by CanTrcv ..... 14
Table 3-4 Features provided beyond the AUTOSAR standard ..... 14
Table 3-5 Service IDs ..... 19
Table 3-6 Errors reported to DET ..... 19
Table 4-1 Static files ..... 20
Table 4-2 Generated files ..... 20
Table 3 Compiler abstraction and memory mapping ..... 21
Table 6-1 CanTrcv_30_ Tja1040_InitMemory ..... 25
Table 6-2 CanTrcv_30_ Tja1040_Init ..... 26
Table 6-3 CanTrcv_30_ Tja1040_SetOpMode ..... 27
Table 6-4 CanTrcv_30_ Tja1040_GetOpMode ..... 28
Table 6-5 CanTrcv_30_ Tja1040_GetBusWuReason ..... 29
Table 6-6 CanTrcv_30_ Tja1040_SetWakeupMode ..... 30
Table 6-7 CanTrcv_30_ Tja1040_GetVersionInfo ..... 31
Table 6-8 CanTrcv_30_Tja1040_CheckWakeup ..... 32
Table 6-9 CanTrcv_30_Tja1040_MainFunction ..... 33
Table 6-10 Services used by the CANTRCV ..... 33
Table 8-1 Glossary ..... 36
Table 8-2 Abbreviations ..... 36

# 1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

| Component Version | | New Features |
|---|---|---|
| CANTRCV_30_TJA1040_SW_MAJOR_VERSION<br>CANTRCV_30_TJA1040_SW_MINOR_ VERSION | 4<br>0 | MICROSAR Identity Manager using Post-build selectable (see Table 3-4) |
| CANTRCV_30_TJA1040_SW_MAJOR_VERSION<br>CANTRCV_30_TJA1040_SW_MINOR_ VERSION | 4<br>2 | ASIL-D support added. The driver can be used in an ASIL-D environment. |

Table 1-1 Component history

# 2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CANTRCV as specified in [1].

| | | |
|---|---|---|
| **Supported AUTOSAR Release*:** | 4.0.3 | |
| **Supported Configuration Variants:** | pre-compile | |
| **Vendor ID:** | CANTRCV_30_TJA1040_VENDOR_ID | 30 decimal<br>(= Vector-Informatik, according to HIS) |
| **Module ID:** | CANTRCV_30_TJA1040_MODULE_ID | 70 decimal<br>(according to ref. [3]) |
| **Instance ID:** | CANTRCV_30_TJA1040_INSTANCE_ID | Specified as pre-compile parameter in the Generation Tool. |

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

“The CAN transceiver driver module is responsible for handling the CAN transceiver hardware chips on an ECU”[1]. The function of a CAN transceiver is to adapt the analogue signal levels of the CAN bus to the logical (digital) ones used by a microcontroller.

Furthermore a CAN transceiver may be able to detect wake-up events, “electrical malfunctions like wiring issues, ground offsets or transmission of long dominant signals”[1]. Depending on provided HW-interface e.g. DIO or SPI the “detected error is summarized by a single port pin”[1] or detailed in a register within the CAN transceiver which can be read by SPI.

[1] [1] p. 7

## 2.1 Architecture Overview

The following figure shows where the CANTRCV is located in the AUTOSAR architecture.

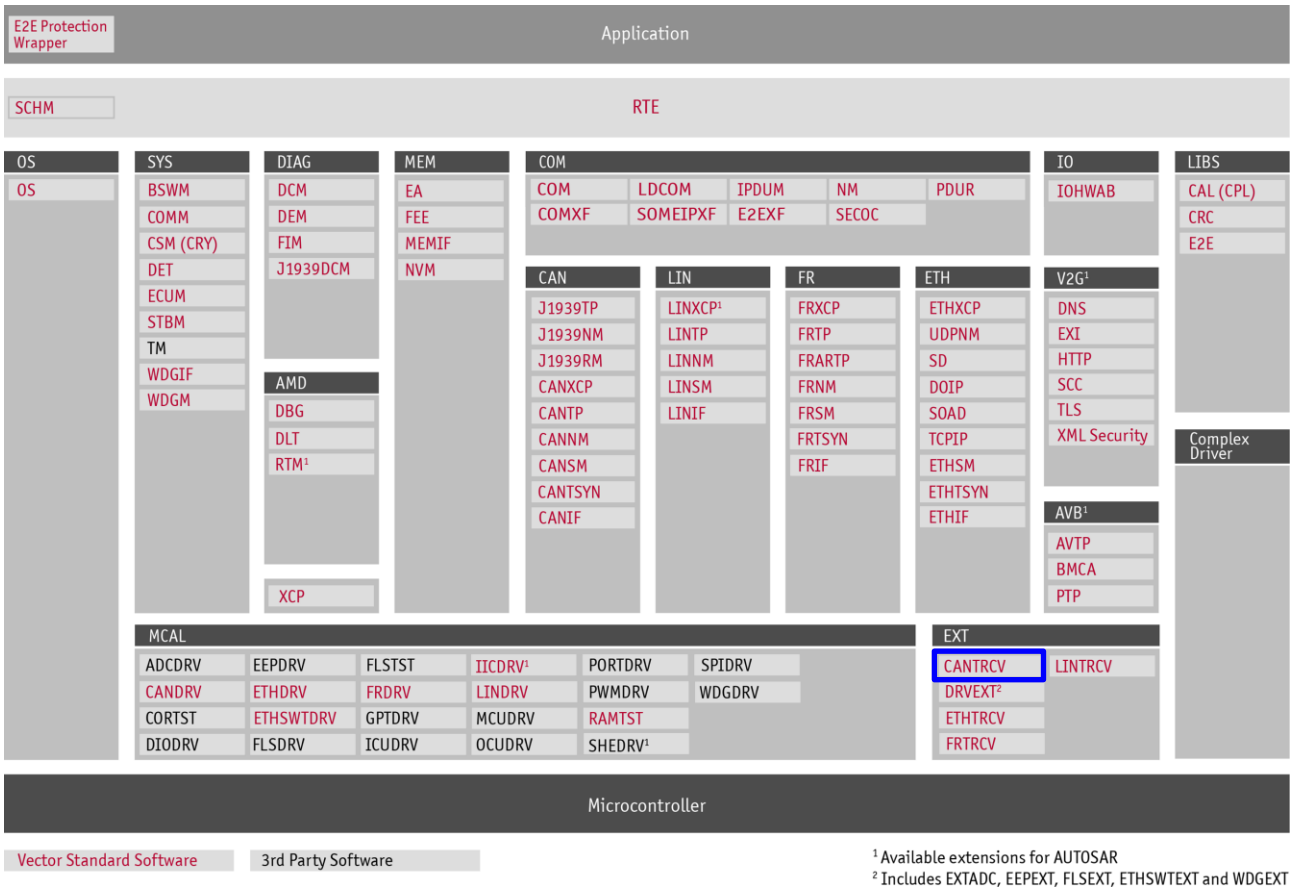



Figure 2-1 AUTOSAR 4.2 Architecture Overview

The next figure shows the interfaces to adjacent modules of the CANTRCV. These interfaces are described in chapter 6.

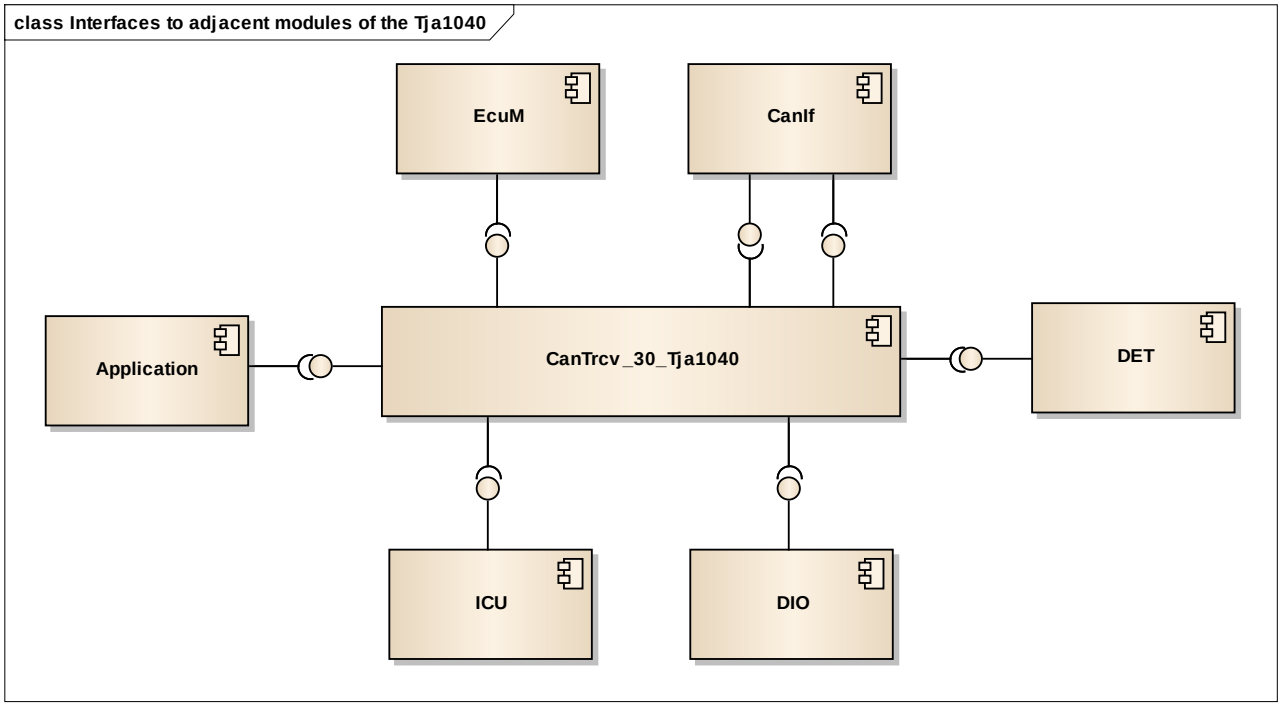



Figure 2-2 Interfaces to adjacent modules of the CANTRCV

## 2.2 Supported Hardware

Although the APIs include the infix `Tja1040` this driver supports several kinds of hardware which are listed in the following:

- NXP Tja1040
- NXP Tja1042
- NXP Tja1044T/GT
- NXP Tja1046
- NXP Tja1049
- NXP Tja1059
- NXP Tja1902A [Only CAN1 of this Multi-Chip Module can be controlled by this driver]
- AMIS 42665
- TI SN65HVDA541
- TI SN65HVD1040-Q1
- TI SN65HVD251
- TI HVDA551-Q1
- TI HVDA553-Q1

- TI TCAN1042V-Q1
- Renesas R2A25416SP
- ON Semiconductor NCV7342
- ON Semiconductor NCV7344
- ON Semiconductor NCV7349
- Microchip MCP2561/2FD
- Freescale Semiconductor MC33901WEF
- Infineon TLE6251D
- Infineon TLE7251
- Infineon TLE8251VSJ
- Infineon TLE9251VLE/VSJ



**Caution**
When using the CAN transceiver to detect remote wake-up events please consider chapter 7.3.1 for proper functionality of this driver.



**Caution (affects only R2A25416SP)**
There is NO waiting time implemented in CAN transceiver driver after transition to standby mode. (See chapter: “Bus Wakeup Detection during Bus Failure Mode” in [11])



**Caution (affects only TJA1902A)**
The TJA1902A contains three fully independent transceivers in a single Multi-Chip Module (MCM). With this driver only one of them (CAN1) is supported.

# 3 Functional Description

## 3.1 Features

The features listed in the following tables cover the complete functionality specified for the CANTRCV.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further CANTRCV functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-4 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported:

| Supported AUTOSAR Standard Conform Features |
|---|
| CAN transceiver initialization |
| CAN transceiver control via DIO |
| Detection of wake-up |
| Getting and setting operating mode of CAN transceiver |

Table 3-1 Supported AUTOSAR standard conform features

### 3.1.1 Deviations

The following features specified in [1] are not supported:

| Not Supported AUTOSAR Standard Conform Features |
|---|
| CAN transceiver control via SPI |
| CAN transceiver self diagnostics |

Table 3-2 Not supported AUTOSAR standard conform features due to hardware limitation

The following chapters list further AUTOSAR-requirements which are not fulfilled or fulfilled only with deviation in order to ensure proper functionality at all.

#### 3.1.1.1 Notification functions

According to SWS [1] the transceiver shall call notification functions of the CANIF. Due to fact that the given `CanTrcvChannelId` is valid only for one transceiver driver instance it was decided to call the following notification function instead:

| SWS API | Used API |
|---|---|
| CanIf_TrcvModeIndication | CanIf_30_Tja1040_TrcvModeIndication |

Table 3 Deviation of APIs used by CanTrcv

Within these function the recalculation of the given CanTrcvIndex has to be performed so that the local index of the driver matches the global index of the CANIF (see parameter CanIfTrcvId).

Affected requirements: CanTrcv086, CanTrcv222, CanTrcv239, CanTrcv238.

#### 3.1.1.2 Unused BSWMD parameters

According to SWS [1] the parameters CanTrcvSPICommRetries and CanTrcvSPICommTimeout must have the multiplicity 1. Due to fact that the SWS does not describe how to use these parameters it was decided to set multiplicity to 0..1 so that they do not have to be used.

Affected requirements: CanTrcv172, CanTrcv171.

#### 3.1.1.3 Initialization

According to SWS [1] each CAN transceiver channel shall be initialized to operating mode which is configured by parameter CanTrcvInitState. Independent of configuration each CAN transceiver channel is initialized into operating mode NORMAL. This is required in order to ensure a reliable initialization of the CAN driver which is processed directly after initialization of the CAN transceiver driver. Mostly a CAN controller requires a recessive level on the Rx-pin or at least that the CAN bus level is reflected on the Rx-pin for a succesful initialization. This can be fullfilled only if the CAN transceiver is in operating mode NORMAL.

Affected requirements: CanTrcv100, CanTrcv148.

### 3.1.2 Additions / Extensions

The following features are provided beyond the AUTOSAR standard:

| Features Provided Beyond The AUTOSAR Standard |
|---|
| MICROSAR Identity Manager using Post-build selectable |

Table 3-4 Features provided beyond the AUTOSAR standard

### 3.1.3 Limitations

#### 3.1.3.1 Support of SPI

This CAN transceiver driver does not support the connection to a CAN transceiver via SPI.

## 3.2 Initialization

After power-on or a reset the CAN transceiver hardware has to be initialized. Therefore the CAN transceiver driver provides two service functions `CanTrcv_30_Tja1040_InitMemory` and `CanTrcv_30_Tja1040_Init`.
The function `CanTrcv_30_Tja1040_InitMemory` initializes all necessary values for the CAN transceiver driver. This function has to be called first after a power-on or a reset.
The function `CanTrcv_30_Tja1040_Init` initializes all CAN transceiver channels which are configured and enabled within the configuration tool. During the initialization each CAN transceiver channel is switched to the operating mode NORMAL.



**Caution**
The function `CanTrcv_30_Tja1040_InitMemory` has to be called before the function `CanTrcv_30_Tja1040_Init` is called.

If a wake-up event was pending before the initialization the CAN transceiver driver stores this event and notifies it by calling of function `EcuM_SetWakeupEvent`.

## 3.3 Operating modes

The CAN transceiver supports only two operating modes:

1. NORMAL and
2. STANDBY.

In operating mode NORMAL the CAN transceiver can receive and transmit data via the CAN-bus. The operating mode STANDBY is a LOW-power mode in which the CAN transceiver is not able to receive correctly and to transmit data at all. The following figure shows the possible mode transitions.

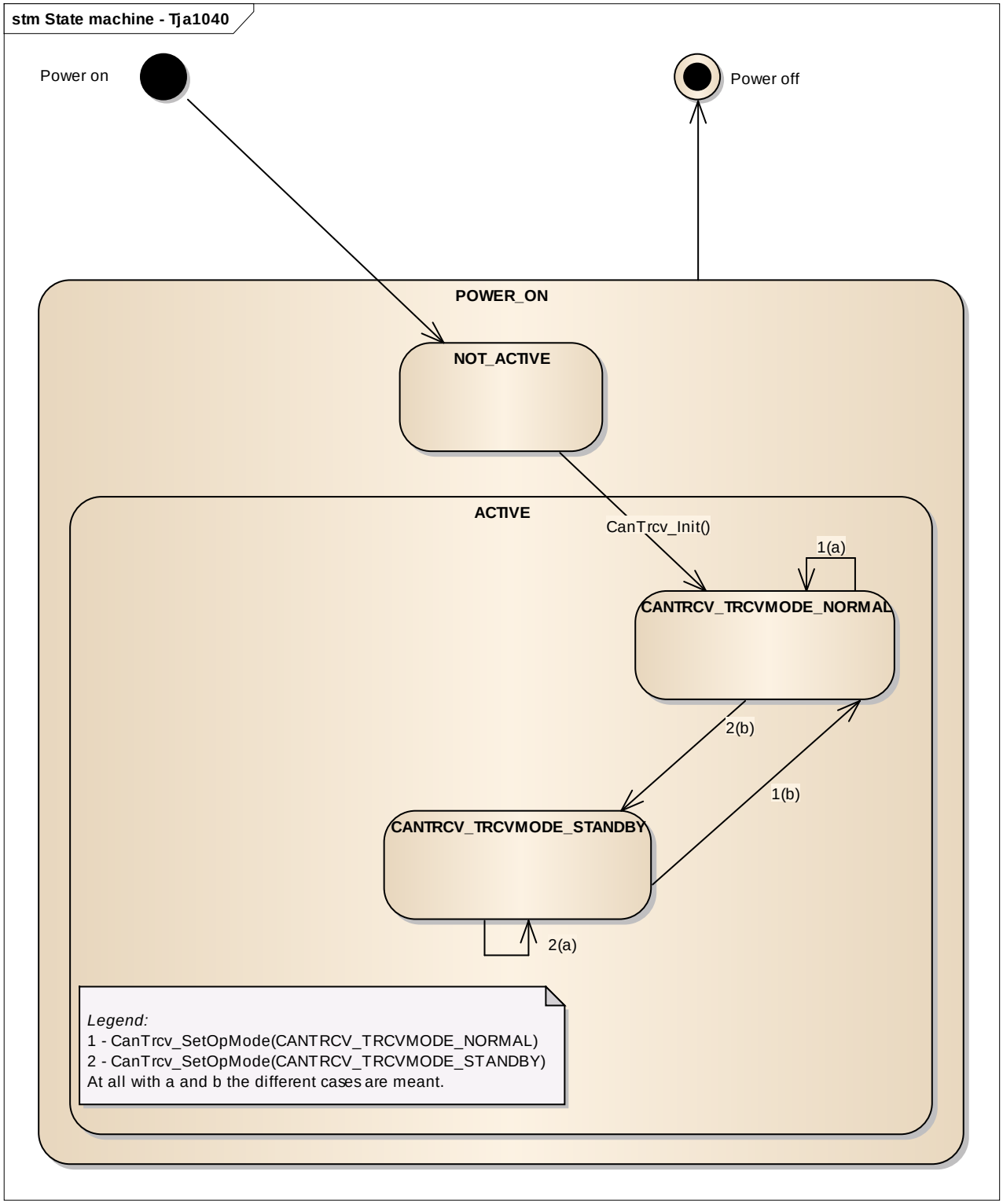



Figure 3-1 Operating mode state machine

The following chapters describe which APIs are to be used in order to set or to get an operating mode.

### 3.3.1 Setting of operating mode

The operating mode of a CAN transceiver channel can be changed by function `CanTrcv_30_Tja1040_SetOpMode`. The operating mode can be changed from NORMAL to STANDBY and vice versa (see Figure 3-1). Note that this CAN transceiver does not support the operating mode SLEEP. If the function is called with the same operating mode which is already set no mode change occurs and `E_OK` is returned.

### 3.3.2 Getting of operating mode

To retrieve the current operating mode of a CAN transceiver channel the function `CanTrcv_30_Tja1040_GetOpMode` can be used.

## 3.4 Main Functions

The CAN transceiver driver provides only one main function: `CanTrcv_30_Tja1040_MainFunction`. This function can be used to check periodically if a wake-up event was detected by the underlying CAN transceiver hardware. This service has only an effect if wake-up support is set to polling (See configuration parameter: `CanTrcvWakeUpSupport`). For further information please see chapter 6.1.9.

## 3.5 Get version info

The function `CanTrcv_30_Tja1040_GetVersionInfo` can be used to get versions of the CAN transceiver driver. This function must be enabled in the configuration tool by enabling the parameter `CanTrcvGetVersionInfo`.
The versions of the CAN transceiver driver can be accessed in two different ways: either by using the function `CanTrcv_30_Tja1040_GetVersionInfo` which returns the versions via the type `Std_VersionInfoType` or by accessing the version defines directly which are specified in the header file `CanTrcv_30_Tja1040.h`. At all following versions and data respectively can be accessed:

Vendor and module-ID:

`CANTRCV_30_TJA1040_VENDOR_ID`

`CANTRCV_30_TJA1040_MODULE_ID`

AUTOSAR revision:

`CANTRCV_30_TJA1040_AR_RELEASE_MAJOR_VERSION`

`CANTRCV_30_TJA1040_AR_RELEASE_MINOR_VERSION`

`CANTRCV_30_TJA1040_AR_RELEASE_PATCH_VERSION`

Software version:

CANTRCV_30_TJA1040_SW_MAJOR_VERSION

CANTRCV_30_TJA1040_SW_MINOR_VERSION

CANTRCV_30_TJA1040_SW_PATCH_VERSION

## 3.6 Wake-up by bus event detection

If wake-up by bus detection is enabled in the configuration the callback function CanTrcv_30_Tja1040_CheckWakeup must be called by the lower layer in order to check or to confirm a potential wake-up event. This function checks the specified CAN transceiver channel. If the specified CAN transceiver channel is in operating mode STANDBY and a wake-up by bus event was detected the CAN transceiver driver calls the API EcuM_SetWakeupEvent in order to confirm a detected wake-up event.



**Caution**
Due to Hardware limitations a wake-up is only detected if a dominant bus level is stable for at least $t_{bus}$ (typically 1.75µs). Refer to the hardware manual for details.

In addition the wake-up reason can be determined by using of an further API (see chapter 3.6.1) or the wake-up detection can be controlled at all (see chapter 3.6.2).

### 3.6.1 Wake-up reason

The function CanTrcv_30_Tja1040_GetBusWakeupReason returns the reason which caused the wake-up.

### 3.6.2 Wake-up mode

The function CanTrcv_30_Tja1040_SetWakeupMode can be used in order to enable or to disable the wake-up reporting at all or just to clear an already reported wake-up event.

## 3.7 Error Handling

### 3.7.1 Development Error Reporting

By default development errors are reported to DET by using of service Det_ReportError() as specified in [2] if development error reporting is enabled (pre-compile parameter CANTRCV_30_TJA1040_DEV_ERROR_DETECT == STD_ON).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service Det_ReportError().

The reported CANTRCV ID is 70. Furthermore the reported CANTRCV instance ID has to be specified in configuration tool. For details please see chapter 7.1.

The reported service IDs identify the services which are described in 6.1. The following table presents the service IDs and the related services:

| Service ID | Service |
|---|---|
| 0x00 | CanTrcv_30_Tja1040_Init |
| 0x01 | CanTrcv_30_Tja1040_SetOpMode |
| 0x02 | CanTrcv_30_Tja1040_GetOpMode |
| 0x03 | CanTrcv_30_Tja1040_GetBusWuReason |
| 0x05 | CanTrcv_30_Tja1040_SetWakeupMode |
| 0x07 | CanTrcv_30_Tja1040_CheckWakeup |
| 0x04 | CanTrcv_30_Tja1040_GetVersionInfo |
| 0x06 | CanTrcv_30_Tja1040_MainFunction |

Table 3-5 Service IDs

The errors reported to DET are described in the following table:

| Error Code | | Description |
|---|---|---|
| 0x01 | CANTRCV_30_TJA1040_E_INVALID_TRANSCEIVER | Invalid channel index is used in function argument list. |
| 0x02 | CANTRCV_30_TJA1040_E_PARAM_POINTER | Invalid pointer (NULL_PTR) is used in function argument list. |
| 0x11 | CANTRCV_30_TJA1040_E_UNINIT | CAN transceiver hardware is not initialized. |
| 0x23 | CANTRCV_30_TJA1040_E_PARAM_TRCV_WAKEUP_MODE | The requested wake-up mode is not valid. |
| 0x24 | CANTRCV_30_TJA1040_E_PARAM_TRCV_OPMODE | The requested operating mode is not supported by the CAN transceiver. |
| 0x40 | CANTRCV_30_TJA1040_E_NO_TRCV_CONTROL | The CAN transceiver is not under control. In such case, the transceiver is in an invalid state. |

Table 3-6 Errors reported to DET

# 4 Integration

This chapter gives necessary information for the integration of the MICROSAR CANTRCV into an application environment of an ECU.

## 4.1 Scope of Delivery

The delivery of the CANTRCV contains the files which are described in the chapters 4.1.1 and 4.1.2:

### 4.1.1 Static Files

| File Name | Description |
|---|---|
| `CanTrcv_30_Tja1040.h` | Header file which has to be included by higher layers. |
| `CanTrcv_30_Tja1040.c` | Implementation |

Table 4-1 Static files

### 4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [config tool].

| File Name | Description |
|---|---|
| `CanTrcv_30_Tja1040_Cfg.h` | Header file contains type definitions and external data declarations. It is included by `CanTrcv_30_Tja1040.h`. |
| `CanTrcv_30_Tja1040_Cfg.c` | Contains the configuration data. |
| `CanTrcv_GeneralTypes.h` | Header file with all CanTrcv types specified by SWS. This file can be included by `Can_GeneralTypes.h`. |

Table 4-2 Generated files

## 4.2 Critical Sections

The CAN transceiver driver calls service functions of upper layers in order to prevent interruption when accessing the CAN transceiver pins.

These service functions have to be provided by the components VStdLib, Schedule Manager or OSEK OS depending on which of these components is used for interrupt disable / restore handling. The component for interrupt control handling has to be selected in the configuration tool.

### 4.2.1 CANTRCV_EXCLUSIVE_AREA_0

This critical section ensures consistent handling of CAN transceiver hardware via the DIO interface.

> Protects: atomic / consistent reading of DIO-pins and setting of them.

> Used in: `CanTrcv_30_Tja1040_Init()`, `CanTrcv_30_Tja1040_SetOpMode()`, `CanTrcv_30_Tja1040_GetOpMode()`,

`CanTrcv_30_Tja1040_CheckWakeup()` and `CanTrcv_30_Tja1040_MainFunction()`.

> Exclude call of CAN transceiver driver APIs: `CanTrcv_30_Tja1040_Init()`, `CanTrcv_30_Tja1040_SetOpMode()`, `CanTrcv_30_Tja1040_GetOpMode()`, `CanTrcv_30_Tja1040_CheckWakeup()` and `CanTrcv_30_Tja1040_MainFunction()`

AND

In fact the instructions within the exclusive area must be executed atomic without any interruption or delay in between.

> Duration: Used in calls to DIO module – SHORT duration

## 4.3 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions defined for the CANTRCV and illustrates their assignment among each other.

| Compiler Abstraction Definitions / Memory Mapping Sections | CANTRCV_30_TJA1040_VAR | CANTRCV_30_TJA1040_APPL_VAR | CANTRCV_30_TJA1040_CONST | CANTRCV_30_TJA1040_CODE | CANTRCV_30_TJA1040_APPL_CODE |
|---|---|---|---|---|---|
| CANTRCV_30_TJA1040_START_SEC_CODE<br>CANTRCV_30_TJA1040_STOP_SEC_CODE | | | | ■ | ■ |
| CANTRCV_30_TJA1040_START_SEC_CONST_UNSPECIFIED<br>CANTRCV_30_TJA1040_STOP_SEC_CONST_UNSPECIFIED | | | ■ | | |
| CANTRCV_30_TJA1040_START_SEC_VAR_NOINIT_UNSPECIFIED<br>CANTRCV_30_TJA1040_STOP_SEC_VAR_NOINIT_UNSPECIFIED | ■ | ■ | | | |

Table 3 Compiler abstraction and memory mapping

# 5 Dependencies to other components

## 5.1 DIO driver

The CAN transceiver driver performs hardware access by calling service functions of the lower layer component DIO driver:

- Function `Dio_WriteChannel` is used to set the logical level of the channel pins the CAN transceiver hardware is connected to.
- Function `Dio_ReadChannel` is used to get the logical level of the channel pins the CAN transceiver hardware is connected to.
- The DIO driver has to provide the pin assignment for the CAN transceiver hardware pins RXD and STB. These pins are referred by the CAN transceiver driver using the symbolic names specified in the configuration tool.

## 5.2 ICU driver

The CAN transceiver driver performs hardware access by calling service functions of the lower layer component ICU driver:

- Function `Icu_DisableNotification()` is used by the transceiver driver to disable the ICU notification after wake-up event notification.
- Function `Icu_EnableNotification()` is used by the transceiver driver to enable the ICU notification during the transition into the STANDBY mode.



**Please note**
The following chapter applies only to this use case:
- the used CAN controller does not support the feature “wake-up by CAN bus”, i.e. the CAN controller can not detect incoming CAN messages and generate a so-called wake-up-interrupt
- the ECU shall wake up and start its own communication due to detected communication on the CAN bus

The proposal described in this chapter enables the user to support the use-case by using an additional µC I/O port to generate the wake-up information via the ICU driver. This I/ O port is connected either parallel to the CAN Rx-pin or is directly attached to the CAN transceivers ERR-pin (depending on the used CAN transceiver). The following steps have to be done for every CAN channel that shall be able to wake up via the CAN bus:

### 5.2.1 ICU configuration

First of all add an ICU channel:

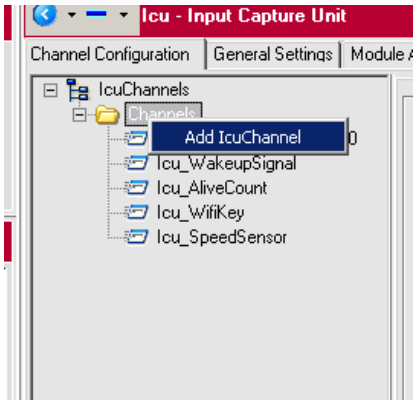



Figure 5-1 Add an ICU channel

The user has to configure the ICU channel and to add a signal notification function.

Additionally the wake-up source for the CAN channel which shall be handled via this ICU channel have to be chosen.

In dependency of the provided ICU configuration options it is possible to configure the `IcuDefaultStartEdge`. This option should be configured to `ICU_FALLING_EDGE`, because the wake-up event is provided by the Rx-pin and / or the ERR-pin via a level change from recessive to dominant.

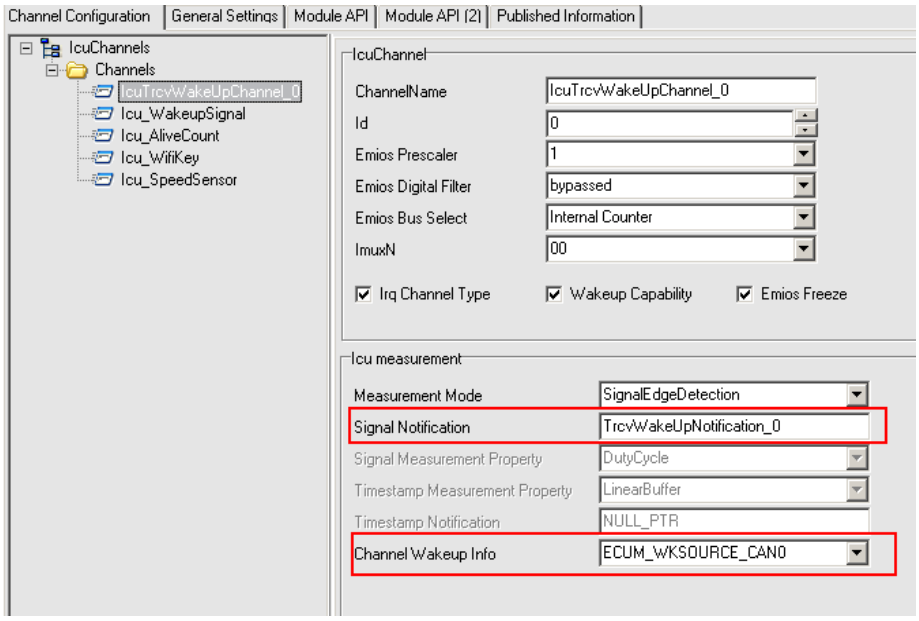



Figure 5-2 ICU channel configuration for wake-up via transceiver

If the transceiver shall also wake the ECU up and not only the CAN channel then it is necessary to activate the `Wakeup Capability` for this ICU channel.

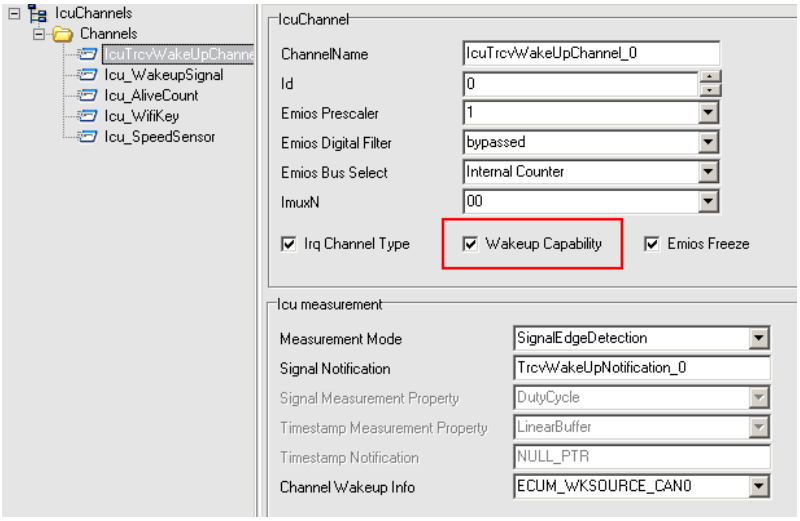



Figure 5-3 ICU wake-up capability

### 5.2.2 Implementation of the signal notification function

The user has to implement the signal notification function as shown in the following example:

**Example**

```
void Icu_TrcvWakeUpNotification_0(void)
{

  /* inform the EcuM about the wakeup event, the parameter
   * is the configured transceiver wakeup source */
  EcuM_CheckWakeup(ECUM_WKSOURCE_CAN0);
}
```

**Please note**

If no wake-up validation is used for the configured wake-up source it is possible to call `EcuM_SetWakeupEvent()` instead of `EcuM_CheckWakeup()`.

# 6 API Description

For an overview of used and provided interfaces please see Figure 2-2.

## 6.1 Services provided by CANTRCV

### 6.1.1 CanTrcv_30_Tja1040_InitMemory

| Prototype | |
|---|---|
| `void` **`CanTrcv_30_Tja1040_InitMemory`** `(void)` | |
| **Parameter** | |
| - | - |
| **Return code** | |
| `void` | none |
| **Functional Description** | |
| This function initializes the memory and needed values of the CAN transceiver driver. | |
| **Particularities and Limitations** | |
| > This function must be called before any other functionality of the CAN transceiver driver.<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function must be called from task level and it is not reentrant. | |

Table 6-1 CanTrcv_30_ Tja1040_InitMemory

### 6.1.2 CanTrcv_30_ Tja1040_Init

| Prototype | |
|---|---|
| `void` **`CanTrcv_30_Tja1040_Init`** `(CanTrcv_30_Tja1040_ConfigType* ConfigPtr)` | |
| **Parameter** | |
| `ConfigPtr[in]` | Pointer to the `CanTrcv_30_Tja1040_Config` struct. If multiple configurations are available the active configuration can be selected by using the related `CanTrcv_30_Tja140_Config_<IdentityName>` structure. |
| **Return code** | |
| `void` | None |
| **Functional Description** | |
| This function initializes all channels of the CAN Transceiver driver which are configured in the configuration tool. This function has the service id 0x00. | |
| **Particularities and Limitations** | |
| > The function `CanTrcv_30_ Tja1040_InitMemory` must be called before the function `CanTrcv_30_Tja1040_Init` can be called.<br>> This function must be called before any other service functionality of the CAN transceiver driver.<br>> The DIO driver must be initialized.<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function must be called from task level and is not reentrant. | |

Table 6-2 CanTrcv_30_ Tja1040_Init

### 6.1.3 CanTrcv_30_Tja1040_SetOpMode

| Prototype | |
|---|---|
| `Std_ReturnType` **`CanTrcv_30_Tja1040_SetOpMode`** `(uint8 CanTrcvIndex, CanTrcv_TrcvModeType OpMode)` | |
| **Parameter** | |
| `CanTrcvIndex[in]` | Index of the selected transceiver channel to which API call has to be applied. |
| `OpMode[in]` | Operating mode which shall be set. |
| **Return code** | |
| `E_OK / E_NOT_OK` | `E_OK`: Change of transceiver operating mode was successful.<br>`E_NOT_OK`: Change of transceiver operating mode failed or passed parameter(s) is/are invalid. |
| **Functional Description** | |
| Sets the CAN transceiver to the requested operating mode. Possible operating modes are:<br>> `CANTRCV_TRCVMODE_NORMAL`<br>> `CANTRCV_TRCVMODE_STANDBY`<br>If return code is E_OK the notification function `CanIf_30_Tja1040_TrcvModeIndication` will be called as soon as the change of operating mode completes. This notification function is called in context of `CanTrcv_30_Tja1040_SetOpMode`.<br>This function has the service id 0x01. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function can be called on task or interrupt level and is only reentrant for different transceiver channels (Parameter: `CanTrcvIndex`). | |

Table 6-3 CanTrcv_30_ Tja1040_SetOpMode

### 6.1.4 CanTrcv_30_ Tja1040_GetOpMode

| Prototype | |
|---|---|
| `Std_ReturnType` **`CanTrcv_30_Tja1040_GetOpMode`** `(uint8 CanTrcvIndex, CanTrcv_TrcvModeType *OpMode)` | |
| **Parameter** | |
| `CanTrcvIndex[in]` | Index of the selected transceiver channel to which API call has to be applied. |
| `OpMode[out]` | Pointer to buffer where the current operating mode is stored. |
| **Return code** | |
| `E_OK / E_NOT_OK` | `E_OK`: Determination of current transceiver operating mode was successful.<br>`E_NOT_OK`: Determination of transceiver operating mode failed or passed parameter(s) is/are invalid. |
| **Functional Description** | |
| Stores the current operating mode of the selected CAN transceiver to variable `OpMode`. Possible operating modes are:<br>> `CANTRCV_CANTRCV_NORMAL`<br>> `CANTRCV_CANTRCV_STANDBY`<br>The mode is determined from status of PINs of the CAN transceiver hardware. This function has the service id 0x02. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> If a change of operating mode was requested (see API: `CanTrcv_30_Tja1040_SetOpMode`) before using this function the reported operating mode may not be valid until the mode change is completed and `CanIf_30_Tja1040_TrcvModeIndication` is called.<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function can be called on task or interrupt level and is reentrant. | |

Table 6-4 CanTrcv_30_ Tja1040_GetOpMode

### 6.1.5 CanTrcv_30_ Tja1040_GetBusWuReason

| Prototype | |
|---|---|
| `Std_ReturnType` **`CanTrcv_30_Tja1040_GetBusWuReason`** `(uint8 CanTrcvIndex, CanTrcv_TrcvWakeupReasonType *Reason)` | |
| **Parameter** | |
| `CanTrcvIndex[in]` | Index of the selected transceiver channel to which API call has to be applied. |
| `Reason[out]` | Pointer to buffer where the bus wake-up reason is stored. |
| **Return code** | |
| `E_OK / E_NOT_OK` | `E_OK`: is returned if the wake-up reason was detected.<br>`E_NOT_OK`: is returned if bus wake-up reason is not detected or feature is not supported. |
| **Functional Description** | |
| Stores the last wake-up reason for the channel `CanTrcvIndex` to `Reason`. Possible wake-up reasons are:<br>> `CANTRCV_WU_INTERNALLY`: The wake-up was caused by setting the CAN transceiver to operating mode NORMAL by using of API: `CanTrcv_30_ Tja1040_SetOpMode`. In this case the function returns `E_OK`.<br>> `CANTRCV_WU_ERROR`: No wake-up was detected by the CAN transceiver and no reason is stored. In this case the function returns `E_NOT_OK`.<br>> `CANTRCV_WU_NOT_SUPPORTED`: No wake-up detection supported by this CAN transceiver. In this case the function returns `E_NOT_OK`.<br>> `CANTRCV_WU_BY_BUS`: The wake-up was caused by an external CAN bus wake-up. In this case the function returns `E_OK`.<br>> `CANTRCV_WU_RESET`: The wake-up was detected after a reset.<br>The wake-up reason is read from state variable. No access to CAN transceiver hardware is performed.<br>This function has the service id 0x03. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> The wake-up reason represents always the last detected wake-up reason. If more than one wake-up event was detected only the last one can be determined by using of this function.<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function can be called on task or interrupt level and is reentrant. | |

Table 6-5 CanTrcv_30_ Tja1040_GetBusWuReason

### 6.1.6 CanTrcv_30_ Tja1040_SetWakeupMode

| Prototype | |
|---|---|
| `Std_ReturnType` **`CanTrcv_30_Tja1040_SetWakeupMode`** `(uint8 CanTrcvIndex, CanTrcv_TrcvWakeupModeType TrcvWakeupMode)` | |
| **Parameter** | |
| `CanTrcvIndex[in]` | Index of the selected transceiver channel to which API call has to be applied. |
| `TrcvWakeupMode[in]` | Requested wake-up mode for the transceiver channel (`CanTrcvIndex`). |
| **Return code** | |
| `E_OK / E_NOT_OK` | `E_OK`: Change of wake-up mode was successful.<br>`E_NOT_OK`: Change of wake-up mode has failed or passed parameter(s) is/are invalid. |
| **Functional Description** | |
| Enables and disables the reporting of wake-up events on channel `CanTrcvIndex`.<br>> `CANTRCV_WUMODE_ENABLE`: Report wake-up events to upper layer.<br>> `CANTRCV_WUMODE_DISABLE`: Do not report wake-up events to upper layer.<br>> `CANTRCV_WUMODE_CLEAR`: Clear a pending wake-up event.<br>This function has the service id 0x05. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> If wake-up handling is not enabled in configuration this function always returns `E_NOT_OK`.<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function can be called on task or interrupt level and is reentrant only for different transceiver channels (Parameter: `CanTrcvIndex`). | |

Table 6-6 CanTrcv_30_ Tja1040_SetWakeupMode

### 6.1.7 CanTrcv_30_ Tja1040_GetVersionInfo

| Prototype | |
|---|---|
| `void` **`CanTrcv_30_ Tja1040_GetVersionInfo`** `(Std_VersionInfoType* VersionInfo)` | |
| **Parameter** | |
| `VersionInfo[out]` | Structure pointer for storing of version info. |
| **Return code** | |
| `void` | None |
| **Functional Description** | |
| Returns the version info of the CAN transceiver module via the parameter `VersionInfo`.<br>This function has the service id 0x04. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> Configuration Variant(s): `CANTRCV_30_TJA1040_GET_VERSION_INFO == STD_ON` | |
| Expected Caller Context | |
| This function can be called on task or interrupt level and is reentrant. | |

Table 6-7 CanTrcv_30_ Tja1040_GetVersionInfo

### 6.1.8 CanTrcv_30_Tja1040_CheckWakeup

| Prototype | |
|---|---|
| `Std_ReturnType` **`CanTrcv_30_Tja1040_CheckWakeup`** `(uint8 CanTrcvIndex)` | |
| **Parameter** | |
| `CanTrcvIndex[in]` | Index of the selected transceiver channel to which API call has to be applied. |
| **Return code** | |
| `E_OK / E_NOT_OK` | `E_OK` a wake-up by bus event was detected.<br>`E_NOT_OK` no wake-up was detected or an error occurred. |
| **Functional Description** | |
| This function requests the CAN transceiver driver to check for wake-ups and to report them. If any wake-up event was detected the CAN transceiver reports it by calling of `EcuM_SetWakeupEvent`.<br>This function has the service id 0x07. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> Do not use the return value `E_OK` for wake-up detection. A wake-up can be considered as valid only if `EcuM_SetWakeupEvent` was called for the corresponding wake-up source.<br>> Configuration Variant(s): - | |
| Call context | |
| This function can be called on task or interrupt level and is reentrant only for different transceiver channels (Parameter: `CanTrcvIndex`). | |

Table 6-8 CanTrcv_30_Tja1040_CheckWakeup

### 6.1.9 CanTrcv_30_Tja1040_MainFunction

| Prototype | |
|---|---|
| `void` **`CanTrcv_30_Tja1040_MainFunction`** `(void)` | |
| **Parameter** | |
| – | - |
| **Return code** | |
| `void` | none |
| **Functional Description** | |
| This service can be used in order to check periodically if a wake-up event was detected by the underlying CAN transceiver hardware.<br>This function has the service id 0x06. | |
| **Particularities and Limitations** | |
| > The CAN transceiver driver must be initialized.<br>> This service has only an effect if wake-up support is set to polling. (See configuration parameter: `CanTrcvWakeUpSupport`)<br>> Configuration Variant(s): - | |
| Expected Caller Context | |
| This function can be called on task level and is not reentrant. | |

Table 6-9 CanTrcv_30_Tja1040_MainFunction

## 6.2 Services used by CANTRCV

In the following table services provided by other components, which are used by the CANTRCV are listed. For details about prototype and functionality refer to the documentation of the providing component.

| Component | API |
|---|---|
| DET | `Det_ReportError` |
| DIO | `Dio_WriteChannel` |
| DIO | `Dio_ReadChannel` |
| CANIF | `CanIf_TrcvModeIndication` |
| ICU | `Icu_EnableNotification` |
| ICU | `Icu_DisableNotification` |

Table 6-10 Services used by the CANTRCV

# 7 Configuration

## 7.1 Configuration with DaVinci Configurator 5

Refer to the integrated online help and parameter descriptions of Configurator 5.

## 7.2 Configuration Variants

The CANTRCV supports the configuration variants

- `VARIANT-PRE-COMPILE`
- `VARIANT-POST-BUILD-SELECTABLE`

The configuration classes of the CANTRCV parameters depend on the supported configuration variants. For their definitions please see the [BSW_specific]_bswmd.arxml file.

## 7.3 Configuration manually in Header Files

Some optional features cannot be configured in the configuration tool. These switches must be set in the user configuration. The following chapters describe the supported options.

### 7.3.1 CANTRCV_30_TJA1040_IMPLEMENT_APPL_CHECK_WA

```
#define CANTRCV_30_TJA1040_IMPLEMENT_APPL_CHECK_WA  STD_ON
```

Normally the pin states are read and evaluated in order to detect a wake-up during initialization or in `CanTrcv_30_Tja1040_CheckWakeup`. This feature has to be used for transceiver whose RXD pin in case of wake-up detection does not remain at dominant bus level while the transceiver is in standby mode e.g. Tja1042.

When enabling this switch, the condition for wake-up detection can be overwritten. If set to `STD_ON`, the macro `CanTrcv_30_Tja1040_Check_Wa(CanTrcvSTB, CanTrcvRXD, CanTrcvChannel)` has to be placed in the user configuration file too. The parameter `CanTrcvSTB` represents the current value of the STB-pin, `CanTrcvRXD` represents the value of the RXD-pin and `CanTrcvChannel` the transceiver channel. This macro has to be implemented as a boolean expression.

**Caution**
When using this feature, it is up to the application to ensure a consistent wake-up detection.

**Example**
The content of the user configuration file could look like this:

```
#define  CANTRCV_30_TJA1040_IMPLEMENT_APPL_CHECK_WA     STD_ON
#define  CanTrcv_30_Tja1040_Check_Wa(CanTrcvSTB, CanTrcvRXD, CanTrcvChannel)
\  (Appl_CanTrcvWakeupOccured(CanTrcvSTB, CanTrcvChannel) == E_OK)
```

© 2018 Vector Informatik GmbH

# 8 Glossary and Abbreviations

## 8.1 Glossary

| Term | Description |
|---|---|
| DaVinci Configurator 5 | Configuration and generation tool for MICROSAR software components |

Table 8-1 Glossary

## 8.2 Abbreviations

| Abbreviation | Description |
|---|---|
| API | Application Programming Interface |
| AUTOSAR | Automotive Open System Architecture |
| BSW | Basis Software |
| CANIF | CAN Interface |
| DET | Development Error Tracer |
| DIO | Digital Input Output |
| EAD | Embedded Architecture Designer |
| ECU | Electronic Control Unit |
| HIS | Hersteller Initiative Software |
| ICU | Input Capture Unit |
| ID | Identifier |
| ISR | Interrupt Service Routine |
| MICROSAR | Microcontroller Open System Architecture (the Vector AUTOSAR solution) |
| RTE | Runtime Environment |
| SRS | Software Requirement Specification |
| SWC | Software Component |
| SWS | Software Specification |

Table 8-2 Abbreviations

# 9 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com